

LISTA DE COTEJO — AUTOEVALUACIÓN

Proyecto Final: ToDo List con API REST
Desarrollo Web en Entorno Servidor · CIFP La Laboral

Estudiante:

Instrucciones: Marca con una X la columna que corresponda. SÍ = implementado y funciona correctamente. PARCIAL = implementado con deficiencias. NO = no implementado. Usa Observaciones para aclaraciones al docente.

• **Ítems en rojo con fondo rosado:** son OBLIGATORIOS para superar la práctica. Un ítem obligatorio marcado como NO supone penalización significativa en la nota final.

En la última hoja del documento hay una tabla donde deberás calificarte en cada uno de los apartados que se desglosan en esta lista de cotejo

1. Arquitectura y organización del proyecto

Max: 1,0 pts

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
El proyecto compila y arranca sin errores.	☐	☐	☐	
La estructura de paquetes sigue una arquitectura MVC o arquitectura vertical (controller, service, repository, model, dto, security...).	☐	☐	☐	
MySQL/Maria DB está configurado como base de datos	☐	☐	☐	
Existe un mecanismo de carga de datos inicial (import.sql). <i>Debe incluir al menos un usuario de cada rol para poder probar los endpoints.</i>	☐	☐	☐	

2. Modelo de datos y persistencia

Max: 1,5 pts

2a. Entidades y relaciones

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
Entidad User con sus atributos, relaciones y cardinalidades.	☐	☐	☐	
Entidad Task con sus atributos, relaciones y cardinalidades.	☐	☐	☐	
Entidad Category con sus atributos, relaciones y cardinalidades.	☐	☐	☐	
Entidad Tag con sus atributos, relaciones y cardinalidades.	☐	☐	☐	
Se han añadido al menos 2 campos adicionales a Task no presentes en el modelo inicial.	☐	☐	☐	

2b. DTOs y justificación

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
Existen DTOs diferenciados para la entrada (crear/editar) y la salida (respuesta). <i>Ejemplo: EditTaskDto para recibir datos, GetTaskDto para devolver. Evita exponer entidades JPA directamente.</i>	☐	☐	☐	
Existe un DTO para User que no expone la contraseña en las respuestas. <i>Ejemplo: UserSummaryDto con id y username, sin password.</i>	☐	☐	☐	
El uso de cada DTO está justificado en comentarios de código <i>El enunciado exige justificar el uso de los DTOs. Esta opción sólo sería obligatoria si se implementa algún DTO.</i>	☐	☐	☐	

3. Endpoints indicados en el enunciado

Max: 3,5 pts

3a. Endpoints de Administrador

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
POST /admin/users/{id}/promote — Promover usuario a GESTOR. <i>Solo accesible con rol ADMIN.</i>	☐	☐	☐	
POST /admin/users/{id}/demote — Degradar GESTOR a usuario. <i>Solo accesible con rol ADMIN.</i>	☐	☐	☐	
GET /admin/users — Listar todos los usuarios.	☐	☐	☐	
GET /admin/users/{id} — Ver un usuario concreto.	☐	☐	☐	
PUT /admin/users/{id} — Editar un usuario.	☐	☐	☐	
DELETE /admin/users/{id} — Eliminar un usuario.	☐	☐	☐	
GET /admin/categories — Listar todas las categorías (admin).	☐	☐	☐	
POST /admin/categories — Crear categoría.	☐	☐	☐	
PUT /admin/categories/{id} — Editar categoría.	☐	☐	☐	
DELETE /admin/categories/{id} — Eliminar categoría.	☐	☐	☐	

3b. Endpoints de Gestor

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
GET /manager/categories — Listar categorías (gestor). <i>Accesible con rol GESTOR o ADMIN.</i>	☐	☐	☐	
POST /manager/categories — Crear categoría.	☐	☐	☐	
PUT /manager/categories/{id} — Editar categoría.	☐	☐	☐	
DELETE /manager/categories/{id} — Eliminar categoría.	☐	☐	☐	

3c. Endpoints de Usuario (2 pts)

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
POST /auth/register — Registro de nuevo usuario (público, sin autenticación).	☐	☐	☐	
GET /task — Listar todas las tareas del usuario autenticado.	☐	☐	☐	
GET /task/{id} — Ver una tarea concreta del usuario conectado	☐	☐	☐	
POST /task — Crear una nueva tarea asociada automáticamente al usuario autenticado.	☐	☐	☐	
PUT /task/{id} — Editar una tarea (solo si es del usuario autenticado).	☐	☐	☐	
DELETE /task/{id} — Eliminar una tarea (solo si es del usuario autenticado).	☐	☐	☐	
GET /task/search?title=... — Buscar tareas por título. <i>El enunciado pide búsqueda por cada campo propio de la tarea.</i>	☐	☐	☐	
GET /task/search?completed=true — Buscar tareas completadas.	☐	☐	☐	

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
GET /task/search?category=... — Buscar tareas por categoría.	☐	☐	☐	
POST /task/{id}/tags — Asignar tags a una tarea.	☐	☐	☐	
DELETE /task/{id}/tags/{tagId} — Eliminar un tag de una tarea.	☐	☐	☐	
GET /task/by-tag?tag=... — Buscar tareas con un tag concreto.	☐	☐	☐	
GET /tag — Listar tags del usuario.	☐	☐	☐	
POST /tag — Crear un tag.	☐	☐	☐	
PUT /tag/{id} — Editar un tag.	☐	☐	☐	
DELETE /tag/{id} — Eliminar un tag.	☐	☐	☐	
GET /categories — Listar categorías disponibles (usuario).	☐	☐	☐	
PUT /user/profile — Modificar perfil de usuario (username, email, password...).	☐	☐	☐	
GET /dashboard — Dashboard con estadísticas de tareas (por categoría, tag, vencidas, creadas hoy...). <i>Endpoint libre en diseño, debe devolver información resumida útil.</i>	☐	☐	☐	

4. Endpoints propios de la ampliación de Task

Max: 1,5 pts

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
Se ha implementado al menos 1 endpoint de búsqueda por cada campo adicional añadido a Task. <i>Ejemplo: si se añadió 'deadline', debe existir GET /task/search?deadline-before=... Si se añadió 'priority', GET /task/search?priority=HIGH.</i>	☐	☐	☐	
Los endpoints de ampliación tienen sentido semántico y son coherentes con los campos añadidos. <i>No se trata de añadir campos sin uso. Cada campo extra debe tener al menos un endpoint asociado.</i>	☐	☐	☐	
Los campos adicionales están correctamente mapeados en la entidad JPA (tipo adecuado, anotaciones JPA). <i>Ejemplo: deadline como LocalDateTime, priority como enum @Enumerated(EnumType.STRING).</i>	☐	☐	☐	

5. Documentación Swagger / OpenAPI

Max: 1,0 pts

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
La dependencia springdoc-openapi-starter-webmvc-ui está en el pom.xml. <i>Versión recomendada: 2.x. La UI estará disponible en /swagger-ui.html.</i>	☐	☐	☐	
20% de los endpoints tienen @Operation con summary y description. <i>Se valorará positivamente si el porcentaje supera el 20%</i>	☐	☐	☐	

6. Seguridad

Max: 1,5 pts

Ítem a verificar	SÍ	PARCIAL	NO	Observaciones
Las dependencias de Spring Security están en el pom.xml (spring-boot-starter-security y spring-security-test).	☐	☐	☐	
Existe una clase SecurityConfig	☐	☐	☐	
La comunicación es Stateless <i>Para APIs REST no se usan sesiones de servidor.</i>	☐	☐	☐	
Los endpoints /auth/register, /swagger-ui/** y /v3/api-docs/** son públicos (sin autenticación).	☐	☐	☐	
El resto de endpoints requieren autenticación.	☐	☐	☐	
El mecanismo de autenticación implementado es Basic Auth	☐	☐	☐	
Los endpoints de usuario comprueban que el recurso pertenece al usuario autenticado (ownership check). <i>Ejemplo: @PreAuthorize("@ownerCheck.check(#id, principal.getId()))" o @PostAuthorize.</i>	☐	☐	☐	
La API devuelve 401 (no autenticado) y 403 (sin permisos) correctamente. <i>Configurar CustomAuthenticationEntryPoint y CustomAccessDeniedHandler.</i>	☐	☐	☐	
Las contraseñas se almacenan hasheadas con BCrypt.	☐	☐	☐	
El mecanismo de autenticación implementado es JWT (alternativo a Basic Auth)	☐	☐	☐	

Resumen de puntuación

Sección	Máx.	Autoevaluación	Calificación docente
1. Arquitectura y organización	1,0	—	—
2. Modelo de datos y persistencia	1,5	—	—
2a. Entidades y relaciones	0,75	—	—
2b. DTOs y justificación	0,75	—	—
3. Endpoints del enunciado	3,5	—	—
3a. Endpoints de administrador	~1,0	—	—
3b. Endpoints de gestor	~0,5	—	—
3c. Endpoints de usuario (base)	~2,0	—	—
4. Endpoints de ampliación (Task)	1,5	—	—
5. Documentación Swagger/OpenAPI	1,0	—	—
6. Seguridad	1,5	—	—
TOTAL	10,0	—	—

⚠ Esta autoevaluación debe entregarse junto con el proyecto.